

Sponsor Follow-Up - Julian's OmniCursor to Omnimarket Bridge

Julian - OmniCursor Intelligence/Docker Track
April 24, 2026

TL;DR

The narrow bridge target from the April 16 sponsor feedback has been delivered:

- One MCP tool named `run_local_review`.
- The MCP tool calls an OmniCursor bridge module.
- The bridge invokes Omnimarket `node_local_review` through the local-first subprocess path:

```
python -m omnimarket.nodes.node_local_review.
```

- Omnimarket is resolved as a local checkout/package via `OMNIMARKET_ROOT`.
- The result is captured, parsed as JSON, and returned back through MCP.

The implementation intentionally did not expand Docker, did not use `onex run`, did not call `omniintelligence` services directly, did not clone Omnimarket from GitHub at runtime, and did not add upstream pattern writes.

In short: OmniCursor can now trigger a real Omnimarket node from Cursor through one MCP tool, using the local-first path the sponsor asked for.

Sponsor Direction Followed

The April 16 feedback redirected the work away from full Docker/runtime parity and toward a smaller local-first bridge. The implemented path follows that direction:

- Primary bridge layer is Omnimarket, not direct `omniintelligence` service APIs.
- First integration target is `node_local_review`.
- Invocation path is subprocess-based `python -m omnimarket.nodes.node_local_review`.
- `OMNIMARKET_ROOT` locates the local Omnimarket checkout.
- Repo-local `omnimarket-main/` is only a local development fallback.
- Docker Compose remains approved as-is, but it was not expanded and is not the primary bridge path.
- `onex run <contract.yaml>` remains out of scope because of the routing validation gaps called out in the sponsor feedback.
- Pattern writes remain local/team-owned; upstream intelligence writes remain out of capstone scope.

What Changed

BRIDGE-B - Repo Hygiene and Bridge Conventions

Purpose: make the repo safer and document the bridge conventions before adding code.

Completed:

- Added local env safety to `.gitignore`:
- `.env`
- `.env.local`
- `.env.*.local`
- Kept local Omnimarket checkout artifacts out of git:
- `omnimarket-main/`
- `omnimarket-main.zip`
- Added an "Omnimarket bridge" section to `CLAUDE.md`.
- Documented `OMNIMARKET_ROOT`.
- Documented that Omnimarket is invoked as a local checkout/package.
- Documented that Omnimarket is never cloned from GitHub at runtime.
- Documented that Docker Compose is approved but not the bridge's primary path.
- Documented that `onex run` and direct `omniintelligence` HTTP calls are out of scope.

BRIDGE-C - Subprocess Bridge to `node_local_review`

Purpose: give OmniCursor a small, typed bridge that can invoke Omnimarket without Docker.

Completed:

- Added `src/omnicursor/omnimarket_bridge.py`.
- Added `run_local_review(...)`.
- Supported:
- `dry_run`
- `max_iterations`
- `required_clean_runs`
- Resolved Omnimarket through:
- `OMNIMARKET_ROOT`
- `repo-local omnimarket-main/` fallback for development only
- Resolved Python through:
- `OMNIMARKET_PYTHON`
- otherwise `sys.executable`
- Ran the subprocess with `cwd=OMNIMARKET_ROOT`.
- Captured `stdout`, `stderr`, return code, command, `cwd`, and Python executable.

- Parsed stdout JSON into a structured BridgeResult.
- Returned structured failures instead of raising exceptions.
- Injected {OMNIMARKET_ROOT}/src into subprocess PYTHONPATH, because Omnimarket uses a src/ layout.
- Used os.pathsep for portable PYTHONPATH handling.

Implementation-time finding:

cwd=OMNIMARKET_ROOT was not enough by itself for `python -m omnimarket.nodes.node_local_review` when Omnimarket is a local source checkout. The subprocess also needs {OMNIMARKET_ROOT}/src in PYTHONPATH unless Omnimarket is installed as a package. The bridge handles this automatically.

BRIDGE-D - MCP Tool

Purpose: expose the bridge to Cursor through one MCP tool.

Completed:

- Added src/omnicursor/mcp/.
- Added src/omnicursor/mcp/omnimarket_bridge_server.py.
- Created one FastMCP("omnicursor-omnimarket") server.
- Registered one MCP tool: run_local_review.
- Tool defaults are safe for discovery:
- dry_run=True
- max_iterations=10
- required_clean_runs=2
- The MCP tool calls omnicursor.omnimarket_bridge.run_local_review.
- The MCP tool returns JSON text to Cursor.
- Added src/omnicursor/mcp/__main__.py so `python -m omnicursor.mcp` can start the server.
- Added .cursor/mcp.json for Cursor MCP registration.
- Added mcp>=1.26.0, <2.0.0 as an optional dependency group in pyproject.toml.

End-to-End Path

```
Cursor
-> MCP over stdio
-> FastMCP server: omnicursor-omnimarket
-> tool: run_local_review
-> omnicursor.omnimarket_bridge.run_local_review(...)
-> subprocess:
    python -m omnimarket.nodes.node_local_review --dry-run
-> Omnimarket node stdout JSON
-> BridgeResult
-> JSON text returned to Cursor
```

Verification Evidence

Baseline before BRIDGE-D:

- 462 passed

After BRIDGE-D:

- 468 passed

Bridge tests:

- tests/test_omnimarket_bridge.py
- 17 passed

MCP tests:

- tests/test_mcp_omnimarket_bridge.py
- 6 passed

Full suite:

- 468 passed

Real bridge smoke:

- OMNIMARKET_ROOT=/Users/jirustaroure/Desktop/OmniCursor/omnimarket-main
- run_local_review(dry_run=True)
- Result: ok: true
- Parsed state included:
- current_phase: "init"
- dry_run: true

Real MCP-level smoke:

- mcp.call_tool("run_local_review", {"dry_run": true})
- Flow reached:
- MCP server
- OmniCursor bridge
- subprocess invocation
- Omnimarket node_local_review
- Result: ok: true
- Parsed state included:
- current_phase: "init"
- dry_run: true

Safety checks:

- No Docker expansion.
- No onex run usage.
- No direct omniintelligence service calls.
- No runtime GitHub clone.
- No upstream pattern write path.

Files Added or Changed

Added:

- src/omnicursor/omnimarket_bridge.py
- tests/test_omnimarket_bridge.py
- src/omnicursor/mcp/__init__.py
- src/omnicursor/mcp/__main__.py
- src/omnicursor/mcp/omnimarket_bridge_server.py
- tests/test_mcp_omnimarket_bridge.py
- .cursor/mcp.json
- docs/plans/2026-04-24-bridge-b-repo-hygiene-plan.md
- docs/plans/2026-04-24-bridge-c-subprocess-bridge-plan.md
- docs/plans/2026-04-24-bridge-d-mcp-tool-plan.md

Changed:

- .gitignore
- CLAUDE.md
- pyproject.toml

How to Corroborate Locally

From the OmniCursor repo:

```
cd /Users/jirustaroure/Desktop/OmniCursor
source .venv/bin/activate

ruff check src/omnicursor/omnimarket_bridge.py \
  src/omnicursor/mcp/ \
  tests/test_omnimarket_bridge.py \
  tests/test_mcp_omnimarket_bridge.py

pytest tests/test_omnimarket_bridge.py tests/test_mcp_omnimarket_bridge.py -v
pytest tests/ -v
```

Direct bridge smoke:

```
OMNIMARKET_ROOT=/Users/jirustaroure/Desktop/OmniCursor/omnimarket-main \
python - &&&'PY'
```

```

from omnicursor.omnimarket_bridge import run_local_review
import json

print(json.dumps(run_local_review(dry_run=True), indent=2))
PY

```

MCP-level smoke:

```

OMNIMARKET_ROOT=/Users/jirustaroure/Desktop/OmniCursor/omnimarket-main \
python - &&&'PY'
import asyncio
import json
from omnicursor.mcp.omnimarket_bridge_server import mcp

async def main():
    content_blocks, _ = await mcp.call_tool("run_local_review", {"dry_run": True})
    print(json.dumps(json.loads(content_blocks[0].text), indent=2))

asyncio.run(main())
PY

```

Cursor setup note:

Cursor may need to be restarted after `.cursor/mcp.json` is added. The MCP command must run in an environment where `omnicursor` and the optional `mcp` dependency are importable. The project `venv` works for this.

What Was Not Chased

The implementation deliberately avoided the paths the sponsor said not to chase:

- No custom Docker image.
- No tighter `omnibase_infra` to `omniintelligence` runtime coupling.
- No direct calls to `intelligence-reducer`, `intelligence-orchestrator`, or `quality-scoring-compute`.
- No `onex run <contract.yaml>`.
- No full `omniintelligence` service parity.
- No upstream pattern write lifecycle.
- No extra MCP tools beyond `run_local_review`.
- No in-process `Omnimarket` handler fallback yet.

Caveats and Next Review Questions

This delivers the first bridge target, but there are a few explicit boundaries:

- The current smoke uses `dry_run=True`, so it validates invocation, MCP transport, subprocess execution, and JSON parsing. It does not claim deeper review-loop semantics beyond what `node_local_review` currently executes in that mode.
- If the next goal is richer review behavior, the next batch should decide whether to deepen `node_local_review` inputs or move to the in-process handler path.

- Docker findings remain separate follow-ups. The bridge path does not depend on Docker.
- Cursor should be tested manually after restart to confirm the MCP tool appears and can be invoked from the Cursor MCP UI.

Ask

Please confirm whether this satisfies the April 16 target:

"Wire one MCP tool to `node_local_review` via subprocess invocation and demonstrate one end-to-end call."

If approved, the next decision is whether to:

1. Harden this MCP demo path for handoff.
2. Add a richer Omnimarket node invocation.
3. Explore the in-process handler path for nodes without `__main__.py`.
4. Defer Docker reproducibility and migration follow-ups to a separate batch.